Weekly Work Review — 2026-08-31

Team Member: @melichristian
Reviewer / QA: @duclairdeugoue
Review Period: Week ending 2026-08-31
Project: Software Development / Training Project
Review Type: Weekly Progress & Engineering Practice Review

---

1. Review Objective

The purpose of this review is not only to evaluate whether work was completed, but also to evaluate **how the work is being organized, documented, implemented, and presented for review**.

The objective is to progressively introduce the engineering practices required when working on a large-scale software project:

- Working from concrete tickets and requirements.
- Following agreed technical approaches.
- Maintaining a clean and consistent project structure.
- Producing traceable deliverables.
- Defining acceptance criteria before implementation.
- Clearly documenting completed work.
- Identifying difficulties and blockers.
- Knowing when a task is actually complete.
- Preparing work so that another developer or QA can review and validate it.

The goal is to progressively move from a learning-oriented workflow toward a **production-oriented engineering workflow**.

---

2. General Assessment

The week shows that work and learning activities took place. However, the work delivered for review was **not sufficiently structured or traceable to a concrete engineering objective**.

The main concern is therefore not the amount of time spent working, but the absence of a clear connection between:

Requirement → Ticket → Implementation → Evidence → Acceptance Criteria → QA → Definition of Done

Without this chain, it becomes difficult for the reviewer to determine:

- What was actually implemented?
- Why was it implemented?
- How was it implemented?
- What decisions were made?
- What difficulties were encountered?
- What remains unfinished?
- How can the work be tested?
- What criteria determine whether the task is complete?

This is an important area to improve before moving toward larger and more complex projects.

---

3. Findings

3.1 Repository Naming Convention

Observation

The GitHub repository name should be reviewed and renamed to follow a consistent naming convention that clearly reflects the subject or purpose of the repository.

Repository names should be:

- Descriptive.
- Consistent with the project's subject.
- Easy to identify.
- Easy to reference from the CLI.
- Free from unnecessary spaces or ambiguous naming.

Expected Practice

The repository name should clearly communicate **what the repository contains or what is being worked on**.

The `README.md` must also be updated after the repository is renamed so that the documentation remains consistent with the new repository identity.

Expected Deliverable

- Repository renamed using an agreed naming convention.
- `README.md` updated accordingly.
- Any references to the old repository name reviewed and updated where necessary.

3.2 Source-Code Directory Naming

Observation

The source-code directory currently contains a space in its name:

`Vue Fundamentals`

This creates unnecessary friction when working from the CLI because paths containing spaces require escaping or quoting.

For example:

```
cd Vue\ Fundamentals/
```

or:

```
cd "Vue Fundamentals/"
```

Although this technically works, it is not the preferred convention for source-code directories.

Expected Practice

Source-code directories should use a predictable naming convention without spaces.

For example:

```
vue-fundamentals/
```

This also aligns with the naming convention commonly generated by npm tooling, where project names are normalized to a kebab-case format.

Expected Deliverable

Rename the directory to follow the agreed convention and verify that:

- CLI commands work without path escaping.
- Project references remain valid.
- `package.json` remains consistent.
- README/documentation references are updated if necessary.

---

4. Main Process Issue: Lack of a Concrete Work Item

The biggest issue identified during this review is that there is currently **no concrete ticket describing what was being worked on during the week**.

The repository was intended to serve as a bank of projects and exercises that can be controlled through clearly defined work items.

However, a professional project cannot rely only on a general list of things to learn or implement.

For example, if we take a project such as the Liongate website:

Project Owner / PM / Developer / Instructor / Intern

@melichristian may currently have several responsibilities:

- Project Owner
- Project Manager
- Software Developer
- Software Instructor
- Software Intern

Reviewer / QA

@duclairdeugoue:

- Code Reviewer
- QA

Because several responsibilities are involved, the work must become **more structured, not less structured**.

A ticket should therefore exist for each concrete piece of work.

For example:

```
Ticket: Implement responsive navigation

Objective:
Implement the responsive navigation behavior for desktop,
tablet and mobile layouts.

Acceptance Criteria:
- Navigation works on desktop.
```

```
- Navigation works on tablet.
- Navigation works on mobile.
- Menu opens and closes correctly.
- No layout overflow occurs.
- Implementation follows the agreed component structure.

Definition of Done:
- Implementation completed.
- Code reviewed.
- Tests completed.
- QA validated.
- Documentation updated where required.
```

This gives both the developer and reviewer a common reference point.

---

5. Missing Weekly Deliverables

The review identified three major gaps.

5.1 No Concrete Work Item

There was no clearly identified ticket that established:

> "This is the specific problem I am solving this week."

Required Improvement

Every work session should begin with a clearly defined work item.

At minimum:

```
Ticket
Objective
Scope
Acceptance Criteria
Definition of Done
```

5.2 No Clearly Demonstrable Work

Although learning and development activities took place during the week, the work was not presented in a way that allowed the reviewer to clearly determine the concrete output produced.

The question should not simply be:

> "Did you work this week?"

The important engineering questions are:

> **What did you do?**
>
> **How did you do it?**
>
> **Why did you choose that approach?**
>
> **What difficulties did you encounter?**
>
> **What did you learn?**
>
> **What remains to be done?**
>
> **What is the next step?**

These questions are important because they develop the ability to communicate technical work, not just perform it.

5.3 No Acceptance Criteria

There were no clearly defined acceptance criteria against which the delivered work could be validated.

Without acceptance criteria, the reviewer cannot objectively determine whether the implementation satisfies the requirement.

The acceptance criteria should be established **before implementation**, or at minimum confirmed before the work begins.

---

6. Missing Definition of Done

Another important issue is the absence of a clear **Definition of Done (DoD)**.

A task should not be considered complete simply because the developer has finished writing code.

Depending on the task, the Definition of Done may include:

- Implementation completed.

- Code formatted and linted.
- Tests added or updated.
- Relevant tests passing.
- Documentation updated.
- Code committed using the agreed convention.
- Pull request created.
- Code review completed.
- QA validation completed.
- Acceptance criteria satisfied.
- No known blocking issues remaining.

The exact DoD can vary depending on the task, but it must be defined.

---

7. Working Agreement

The purpose of our working sessions is to establish a pattern that can eventually be applied independently.

The expected workflow is:

```
1. Propose the work
       ↓
2. Define the objective
       ↓
3. Define the scope
       ↓
4. Define acceptance criteria
       ↓
5. Define Definition of Done
       ↓
6. Get the approach confirmed
       ↓
7. Implement
       ↓
8. Document what was done
       ↓
9. Present evidence
       ↓
10. Code Review
       ↓
11. QA
       ↓
12. Close the ticket
```

The important point is that once the approach has been discussed and confirmed, the implementation should follow that agreed direction unless a new issue requires the approach to be reconsidered.

If the implementation reveals a problem with the original approach, that should be communicated rather than silently changing direction.

---

8. Expected Weekly Work Report

Going forward, every weekly work session should end with a short report following this structure.

Work Completed

What was actually implemented or completed?

```
- ...
- ...
- ...
```

How It Was Done

Briefly explain the technical approach.

```
- ...
- ...
- ...
```

Technical Decisions

Explain important decisions made during implementation.

```
Decision:
Why:
Alternative considered:
```

Difficulties / Blockers

Document anything that slowed down or blocked the work.

```
Problem:
Impact:
Investigation:
Resolution / Current status:
```

Evidence

Provide evidence that allows the reviewer to verify the work.

Examples:

- Commit(s)
- Pull Request
- Screenshots
- Test results
- Logs
- Demo
- Documentation
- Relevant code references

Acceptance Criteria Status

Acceptance Criteria	Status	Evidence
Criterion 1	✅ / ❌	Link / reference
Criterion 2	✅ / ❌	Link / reference
Criterion 3	✅ / ❌	Link / reference

Definition of Done

Requirement	Status
Implementation completed	✅ / ❌
Tests completed	✅ / ❌
Documentation updated	✅ / ❌
Code review completed	✅ / ❌
QA completed	✅ / ❌

Next Step

Clearly state what should happen next.

```
Next action:
Expected outcome:
```

---

9. Expectations for the Next Work Session

Before starting the next implementation cycle, please:

1. Organize the repository and development environment.
2. Apply the agreed repository and directory naming conventions.
3. Create or identify a concrete ticket for the work.
4. Clearly define the objective and scope.
5. Define acceptance criteria.
6. Define the Definition of Done.
7. Propose the implementation approach.
8. Wait for confirmation where the workflow requires it.
9. Execute the confirmed approach.
10. Provide a clear work report at the end of the session.

The goal is not to slow down development with documentation.

The goal is to make the work **visible, reviewable, measurable, and reproducible**.

---

10. Review Decision

Item	Result
Workload	0% accepted
Status	❌ Failed
Reason	Work could not be sufficiently validated against a concrete ticket, acceptance criteria, and Definition of Done
Next Action	Restart the work using the agreed workflow
Deadline	2026-08-31

> **Important:** The "0%" assessment does not mean that no time or effort was spent during the week. It means that **no sufficiently verifiable deliverable could be accepted against the expected engineering workflow**.

---

11. Final Feedback

The intention of this review is not to discourage the work that was done during the week. The objective is to help establish the habits required to work effectively on larger-scale software projects.

You did spend time working and learning during the week. The main improvement required is to make that work **concrete and traceable**.

As an engineer, being able to say:

> "I worked on this."

is not enough.

You should progressively be able to explain:

> **What did I work on?**
> **Why was I working on it?**
> **What approach did I take?**
> **What did I implement?**
> **What problems did I encounter?**
> **How did I solve them?**
> **What evidence proves the work is complete?**
> **What remains to be done?**
> **What is my next action?**

These are the habits that will allow you to transition from working on isolated exercises to contributing effectively to a production-grade engineering team.

For now, please reorganize the work environment and restart the current work following the confirmed workflow.

We can also schedule a meeting to go through this review together, clarify any points that may not be clear, and make sure the expectations for the next working sessions are aligned.

Keep going — the purpose of this process is to help you improve, become more autonomous, and progressively reach the level required to work on larger projects.